# Insurance Management Platform

# Developer Handoff & Build Guide

A step-by-step record of the project setup, architecture, completed work, Git/GitHub workflow, common mistakes, troubleshooting, and the next development roadmap.

## Purpose of this document

This guide is intended for a fresh developer who needs to understand what has been built so far and continue development without losing context. It combines the project requirements supplied for the Insurance Management Platform with the implementation progress completed in this development session. Where a detail was not explicitly confirmed, it is marked as a recommendation or a point to verify rather than presented as a fact.

The project specification calls for a full-stack Insurance Management Platform with role-based access for Administrator, Insurance Agent, and Customer; Customer, Policy, Claim, Premium, Document, and Reports modules; PostgreSQL; Flask/SQLAlchemy/Flask-Migrate; JWT authentication; password hashing; Git/GitHub; and a React frontend. The planned sequence is authentication/database first, followed by business modules, search/pagination, authorization, validation, testing, UI, and deployment.

# 1. Project Vision and Requirements

The application is intended to manage insurance operations through a full-stack system. The requirements identify the following major modules:

• User authentication and role management

• Customer Management

• Policy Management

• Premium Tracking

• Claim Management

• Document Management

• Reports Dashboard

• Search, filters, and pagination

• Validation and error handling

• Frontend UI and integration

• Testing and deployment

## Roles

• ADMIN: administrative access and management capabilities.

• AGENT: insurance-agent workflows such as customer and policy operations.

• CUSTOMER: customer-facing access and personal insurance workflows.

## Required core technologies from the project specification

• Python Flask

• PostgreSQL

• Flask-SQLAlchemy

• Flask-Migrate / Alembic

• JWT-based authentication

• Password hashing

• Git and GitHub

• React frontend

## Important source note

The original requirements document is the authoritative basis for the project scope. This handoff guide records the actual progress reported during development, but a new developer should still inspect the current repository before changing code.

# 2. Current Architecture and Development State

```
insurance-management-platform/
■■■ backend/
■ ■■■ migrations/
■ ■■■ models/
■ ■ ■■■ __init__.py
■ ■ ■■■ user.py
```

```
■ ■ ■■■ customer.py
■ ■ ■■■ policy.py
■ ■■■ routes/
■ ■■■ schemas/
■ ■■■ uploads/
■ ■■■ venv/
■ ■■■ .env
■ ■■■ app.py
■ ■■■ config.py
■ ■■■ requirements.txt
■■■ frontend/
■■■ .gitignore
■■■ .git/
```

The exact contents of every current source file were not available in this session, so a new developer should treat this tree as the working project structure and inspect the repository for the authoritative implementation.

# 3. Environment Setup

The development environment uses Windows PowerShell. The backend runs as a Flask application and the database is PostgreSQL.

## Typical virtual environment activation command

```
.\venv\Scripts\Activate.ps1
```

## If the virtual environment is inside backend:

```
cd backend
.\venv\Scripts\Activate.ps1
```

## Run the Flask application

```
python app.py
```

The application was successfully run during development. The PostgreSQL connection was also confirmed as working.

## Environment variables

The project uses a .env file for secrets and database credentials. The real .env must never be committed to GitHub.

```
DATABASE_URL=postgresql+psycopg://postgres:YOUR_PASSWORD@localhost:5432/insurance_db
SECRET_KEY=your-secret-key
JWT_SECRET_KEY=your-jwt-secret-key
```

Use real values only in the local .env file. Share a .env.example containing placeholders, not real credentials.

# 4. PostgreSQL and Database Setup

A PostgreSQL database named insurance_db was created and the Flask application was configured to connect to it through DATABASE_URL.

The database layer uses SQLAlchemy. Flask-Migrate/Alembic is used to manage schema changes.

## Typical migration workflow

```
flask db migrate -m "Describe the schema change"
flask db upgrade
```

For a new developer, always inspect the generated migration before applying it. Never assume an autogenerated migration is correct without reviewing it.

# 5. Flask-Migrate Setup

Flask-Migrate was configured successfully. The purpose is to track database schema changes in migration files instead of manually recreating tables.

Expected workflow after changing a SQLAlchemy model:

• Change the model.

• Run flask db migrate with a clear message.

• Review the generated migration.

• Run flask db upgrade.

• Verify the database schema.

• Commit the model and migration together to GitHub.

# 6. User Model and Authentication

The User model was created with the project-required fields: id, name, email, password, and role. The users table was created through migration.

## Security rules

• Passwords must never be stored as plain text.

• Use a password hashing library such as Flask-Bcrypt for password hashing and verification.

• Do not return password hashes from API responses.

• Keep SECRET_KEY, JWT_SECRET_KEY, DATABASE_URL, and other credentials out of GitHub.

## Registration

Completed. The registration API validates required fields, checks email duplication, hashes the password, stores the user, and returns safe user information without the password.

## Login + JWT

Completed. The login flow validates credentials and returns a JWT access token. The role is included in JWT claims for authorization checks.

## Protected routes and role authorization

Completed. JWT-protected endpoints require a valid token, and reusable role authorization restricts access according to ADMIN, AGENT, and CUSTOMER roles.

# 7. Customer Management Progress

The Customer SQLAlchemy model and migration were completed. The Customer fields follow the project specification: id, name, dob, phone, address, and email.

## Customer creation API

Completed. The create-customer endpoint is JWT protected and restricted to ADMIN and AGENT. Validation was added for required fields, date, email, phone, and duplicate email according to the implemented model rules.

## Next Customer tasks

• Get all customers.

• Get customer by ID.

• Update customer.

• Delete customer if permitted by business rules.

• Search customers.

• Pagination and filtering.

• Customer history and relationships to policies/claims as the data model evolves.

# 8. API Testing Workflow

Postman was used to test the authentication and customer APIs.

## Typical JWT workflow

• Register a user.

• Log in with email and password.

• Copy the access_token.

• In Postman, use Authorization → Bearer Token.

• Send the token with protected requests.

• Test both allowed and forbidden roles.

## Important test cases already used or expected

• Successful registration.

• Duplicate registration email.

• Successful login.

• Wrong password.

• Unknown email.

• Protected route without token.

• Protected route with invalid token.

• Allowed role.

• Forbidden role.

• Customer creation with valid data.

• Customer creation with missing fields.

- Invalid email/date/phone.
- Duplicate customer email.

# 9. Git and GitHub Workflow

GitHub was introduced after the core authentication milestone. The local project was initialized as a Git repository and connected to the GitHub repository shra73/insurance-management.

The local folder and GitHub repository names do not need to match. The local folder is insurance-management-platform while the GitHub repository is insurance-management.

## Important Git setup

```
git init
git remote add origin https://github.com/shra73/insurance-management
git remote -v
```

The repository was pushed to the main branch after the initial commit.

## Recommended feature workflow

```
git status
git add .
git status
git commit -m "Describe the completed feature"
git push
```

Commit only after a complete feature is implemented and tested. Do not commit every line of code. Use meaningful feature-level commits.

## Known Git milestones

• Initial project setup with Flask and PostgreSQL

• Implement authentication and role-based authorization

• Add customer database model

• Add customer creation API

The final customer-creation commit was the immediate next Git milestone to push at the point this document was requested; verify the GitHub repository to confirm whether it has already been pushed.

## .gitignore essentials

```
venv/
env/
backend/venv/
.env
backend/.env
frontend/node_modules/
instance/
__pycache__/
*.pyc
.vscode/
```

A critical verification was performed with git status --ignored. backend/venv and frontend/node_modules were confirmed ignored, and backend/.env was subsequently added to the ignored list. Never use git add -f on secrets.

# 10. Problems Faced and Solutions

## Problem: User model import error

The user.py file was empty, while app.py attempted to import a lowercase user object. This caused an ImportError.

Solution: Create a proper User SQLAlchemy class and use the correct case-sensitive import, for example from models.user import User. Python names are case-sensitive.

### Problem: Git repository was not initialized

Running git status produced: fatal: not a git repository.

Solution: Run git init in the project root, then add the GitHub remote and verify with git remote -v.

### Problem: Virtual environment and node_modules could be accidentally committed

Solution: Add backend/venv/ and frontend/node_modules/ to .gitignore and verify using git status --ignored.

### Problem: Secrets could be accidentally uploaded

Solution: Add .env and backend/.env to .gitignore. Use .env.example with placeholder values only. Always inspect git status before git add . and before committing.

### Problem: GitHub repository was initially empty

Solution: Create an empty GitHub repository without automatically adding README, .gitignore, or license when connecting an existing local project. Then connect the local repository with git remote add origin.

### Problem: Unclear commit timing

Solution: Commit after a complete, tested milestone. The recommended sequence is implement → test → review git status → commit → push.

## 11. Basic Mistakes New Developers Should Avoid

• Do not store plain-text passwords.

• Do not commit .env files or real API/database credentials.

• Do not commit virtual environments or node_modules.

• Do not run migrations without reviewing generated migration files.

• Do not change database models without creating and applying a migration.

• Do not build all modules at once. Complete one feature, test it, commit it, then continue.

• Do not trust a 200 response alone; test validation, authorization, duplicate data, and error paths.

• Do not return password hashes, secrets, or internal exception details in API responses.

• Do not bypass JWT protection on endpoints that contain private or role-restricted data.

• Do not assume role authorization is correct without testing each role separately.

• Do not use unclear Git commit messages such as 'changes' or 'update'.

• Do not manually edit production database schemas when migrations are the project standard.

## 12. Current Completion Checklist

| Area | Status |
| --- | --- |
| Flask setup | Done |
| PostgreSQL | Done |
| SQLAlchemy | Done |
| Flask-Migrate | Done |

| Area | Status |
|---|---|
| User model/table | Done |
| Registration | Done |
| Password hashing | Done |
| Login + JWT | Done |
| JWT protected routes | Done |
| Role-based authorization | Done |
| Customer model/table | Done |
| Create Customer API | Done |
| Get customers | Next |
| Customer by ID | Pending |
| Update/Delete Customer | Pending |
| Search/Pagination | Pending |
| Policy Management | Pending |
| Premium Tracking | Pending |
| Claim Management | Pending |
| Document Management | Pending |
| Reports Dashboard | Pending |
| Testing | Pending |
| React frontend | Pending |
| Frontend integration | Pending |
| Deployment | Pending |

# 13. Recommended Next Development Sequence

1. Finish and push the Customer Create API milestone if not already pushed.
2. Implement Get All Customers with JWT protection, role checks, pagination, and clean response formatting.
3. Implement Get Customer by ID.
4. Implement Update Customer.
5. Implement Delete Customer according to business rules.
6. Add customer search and filters.
7. Build Policy model and migration.
8. Build Policy CRUD and customer-policy relationships.
9. Build Premium Tracking.
10. Build Claim Management and claim status workflow.
11. Build Document Management and secure file handling.
12. Build Reports Dashboard.
13. Add validation and centralized error handling.
14. Add comprehensive backend tests.
15. Build React frontend.
16. Integrate frontend with backend APIs and JWT authentication.
17. Deploy backend, PostgreSQL, frontend, and configure production secrets.

# 14. Recommended Working Pattern for Every New Module

```
1. Understand the requirement.
2. Design/update the database model.
3. Create a Flask-Migrate migration.
4. Apply and verify the migration.
5. Implement one API endpoint.
6. Add JWT and role checks.
7. Validate inputs.
8. Test success and failure cases in Postman.
9. Review the code and git diff/status.
10. Commit the complete feature.
11. Push to GitHub.
12. Move to the next endpoint.
```

This incremental approach keeps the project understandable, makes debugging easier, and produces a useful GitHub history.

# 15. Fresh Developer Quick Start

```
# 1. Clone the repository
git clone <your-repository-url>

# 2. Enter the project
cd insurance-management-platform

# 3. Create/activate backend virtual environment
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
```

```
# 4. Install dependencies
pip install -r requirements.txt

# 5. Create local .env
# Add DATABASE_URL, SECRET_KEY, JWT_SECRET_KEY

# 6. Make sure PostgreSQL database exists
# Database name: insurance_db

# 7. Apply migrations
flask db upgrade

# 8. Run the application
python app.py
```

The exact commands can vary depending on how the Flask application exposes its CLI entry point. If flask db is not detected, inspect the current app configuration and follow the repository's established Flask-Migrate setup.

# 16. Final Handoff Notes

At the time of this document, the project has a functioning Flask/PostgreSQL foundation, database migrations, user authentication, JWT security, role-based authorization, a Customer model, and customer creation functionality. The next logical task is to complete the remaining Customer CRUD and then proceed through Policy, Premium, Claim, Document, and Reports modules.

A fresh developer should first clone the repository, inspect the current branch and latest commits, create a local .env from .env.example, verify PostgreSQL connectivity, run migrations, start the Flask server, and test the already completed authentication and customer endpoints before making changes.

This document is a development handoff snapshot, not a substitute for the actual source code or the original project requirements. Always treat the current GitHub repository and the requirements document as the source of truth for the latest implementation.